

MySQL GROUP BY Master Guide (Shopping DB)

1. Count how many orders each customer placed

```
SELECT customer_name, COUNT(*) AS total_orders
FROM orders
GROUP BY customer_name;
```

2. Total amount spent by each customer

```
SELECT customer_name, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_name;
```

3. Count products in each category

```
SELECT category, COUNT(*) AS product_count
FROM products
GROUP BY category;
```

4. Average price of products in each category

```
SELECT category, AVG(price) AS avg_price
FROM products
GROUP BY category;
```

5. Maximum product price in each category

```
SELECT category, MAX(price) AS max_price
FROM products
GROUP BY category;
```

6. Group by Date (total orders per day)

```
SELECT order_date, COUNT(*) AS orders_on_day
FROM orders
GROUP BY order_date;
```

7. Group by Multiple Columns

```
SELECT category, name AS product_name, COUNT(*) AS sold_count
FROM products
JOIN order_items ON products.product_id = order_items.product_id
GROUP BY category, name;
```

8. GROUP BY with HAVING - Only show customers who placed more than 1 order

```
SELECT customer_name, COUNT(*) AS total_orders
FROM orders
GROUP BY customer_name
HAVING total_orders > 1;
```

MySQL GROUP BY Master Guide (Shopping DB)

9. Group Orders by Month

```
SELECT MONTH(order_date) AS order_month, COUNT(*) AS total_orders
FROM orders
GROUP BY MONTH(order_date);
```

10. Top-Selling Product (by quantity)

```
SELECT p.name, SUM(oi.quantity) AS total_quantity
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_id, p.name
ORDER BY total_quantity DESC
LIMIT 1;
```

11. Group by + Join + Aggregate: Revenue by Category

```
SELECT p.category, SUM(oi.quantity * oi.price_per_unit) AS revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.category;
```

12. Subquery: Customers who spent more than the average total

```
SELECT customer_name, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_name
HAVING total_spent > (
    SELECT AVG(amount)
    FROM orders
);
```

13. Group by + COUNT DISTINCT

```
SELECT order_date, COUNT(DISTINCT customer_name) AS unique_customers
FROM orders
GROUP BY order_date;
```

14. Group by with Alias in ORDER BY

```
SELECT customer_name, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_name
ORDER BY total_spent DESC;
```

15. Show the total number of products per price range (less than 500, 500-1000, above 1000)

```
SELECT
CASE
```

MySQL GROUP BY Master Guide (Shopping DB)

```
    WHEN price < 500 THEN 'Below 500'
    WHEN price BETWEEN 500 AND 1000 THEN '500-1000'
    ELSE 'Above 1000'
END AS price_range,
COUNT(*) AS total_products
FROM products
GROUP BY price_range;
```

16. Find customers who placed orders on more than 2 different dates

```
SELECT customer_name, COUNT(DISTINCT order_date) AS active_days
FROM orders
GROUP BY customer_name
HAVING active_days > 2;
```

17. Get the average quantity of each product sold across all orders

```
SELECT p.name, AVG(oi.quantity) AS avg_quantity
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.name;
```

18. Find the day with the highest number of orders

```
SELECT order_date, COUNT(*) AS total_orders
FROM orders
GROUP BY order_date
ORDER BY total_orders DESC
LIMIT 1;
```

19. List categories that have more than one product

```
SELECT category, COUNT(*) AS product_count
FROM products
GROUP BY category
HAVING product_count > 1;
```